



Strada Costache Negri nr. 50, 700071, Iași
Tel: +40 332 418 760; Fax: +40 232 218 240
E-mail: contact@cnavais.ro
Web page: www.cnavais.ro

Colegiul Național „Vasile Alecsandri” Iași

Agregator de știri automat bazat pe microservicii

LUCRARE PENTRU ATESTAREA COMPETENȚELOR
PROFESIONALE LA INFORMATICĂ

Elev: Sonea Andrei
Clasa: XII-a B
Profesor coordonator: Rusu Beatris

SESIUNEA MAI 2026

Cuprins

1	Motivația alegerii temei și utilitatea aplicației	2
1.1	Utilitatea aplicației	2
2	Structura aplicației	2
2.1	Organizarea conținutului informațional	2
2.2	Structuri de date utilizate	2
3	Tehnologii de bază utilizate	4
3.1	Limbajul de programare Rust: Siguranță și Performanță	4
3.2	Limbajul de programare Python: Limbajul Inteligenței Artificiale	5
3.3	Biblioteca React: Construirea Interfețelor Moderne	6
3.4	Comunicarea asincronă între servicii folosind NATS JetStream	7
4	Detalii tehnice de implementare	9
4.1	Backend-ul în Rust: Performanță și Securitate	9
4.2	Procesarea ML în Python: Inteligență Asincronă	10
4.3	Frontend-ul în React: Interfață Reactivă și Tipizată	11
5	Resurse hard și soft necesare	12
5.1	Resurse Soft	12
5.2	Resurse Hard	12
6	Modalități de utilizare	12
6.1	Instalare și Pornire	12
6.2	Fluxul Utilizatorului	12
7	Posibilități de dezvoltare	12

1 Motivația alegerii temei și utilitatea aplicației

În era digitală actuală, volumul de informații disponibil online este copleșitor. Utilizatorii se confruntă cu fenomenul de "infobezitate", unde filtrarea conținutului relevant devine o sarcină consumatoare de timp. Redy a fost conceput ca o soluție modernă pentru centralizarea, organizarea și analizarea automată a fluxurilor de știri (RSS/Atom).

1.1 Utilitatea aplicației

Aplicația Redy oferă următoarele avantaje:

- **Centralizare:** Utilizatorul poate urmări zeci de surse de știri într-o singură interfață coerentă.
- **Analiză Automată:** Folosind Machine Learning, aplicația categorizează articolele și analizează sentimentul acestora (pozitiv, negativ, neutru).
- **Eficiență:** Extragerea automată a conținutului complet al articolelor elimină necesitatea de a naviga pe site-uri externe pline de reclame.
- **Personalizare:** Utilizatorii își pot organiza fluxurile în categorii și pot marca articolele favorite.

2 Structura aplicației

Proiectul Redy utilizează o arhitectură de tip microservicii/servicii distribuite, optimizată pentru performanță și scalabilitate.

2.1 Organizarea conținutului informațional

Sistemul este împărțit în patru componente principale:

1. **Serverul Backend (Rust):** Nucleul aplicației, responsabil pentru API-ul REST, autentificare și coordonarea datelor.
2. **Interfața Client (React):** O aplicație Single Page Application (SPA) modernă care oferă o experiență de utilizare fluidă.
3. **ML Worker (Python):** Un serviciu specializat în procesarea limbajului natural (NLP).
4. **Baza de Date și Mesageria:** PostgreSQL pentru stocare și NATS JetStream pentru comunicare asincronă.

2.2 Structuri de date utilizate

Principalele entități stocate în baza de date includ:

- **User:** Informații despre utilizatori și setări.
- **Feed:** Sursa de știri (URL, nume).

- **Article:** Datele brute ale articolelor (titlu, link, conținut HTML).
- **ArticleData:** Metadate generate de procesele de ML (categoria, scorul de sentiment).
- **Category:** Grupări de fluxuri definite de utilizator.

3 Tehnologii de bază utilizate

Această secțiune prezintă limbajele și instrumentele utilizate, explicând de ce acestea reprezintă standardul actual în industrie și cum se raportează ele la conceptele studiate în curricula școlară.

3.1 Limbajul de programare Rust: Siguranță și Performanță

Deși nu este studiat în liceu, Rust este considerat "succesorul spiritual" al limbajelor C și C++. În timp ce C++ oferă programatorului control total asupra memoriei, acesta vine cu riscul unor erori grave (precum accesarea unei zone de memorie inexistente), care pot duce la prăbușirea programului sau la vulnerabilități de securitate.

De ce Rust?

- **Compilerul ca "Profesor":** În Rust, compilerul este mult mai strict decât în C++. Acesta verifică nu doar sintaxa, ci și *logica gestionării memoriei*. Dacă un programator încearcă să facă o operațiune nesigură, programul pur și simplu nu se va compila, oferind mesaje de eroare extrem de detaliate care explică "de ce" nu este corect.
- **Viteză fără compromisuri:** Rust este un limbaj **compilat** (ca și C++), ceea ce înseamnă că este transformat direct în cod mașină pe care procesorul îl înțelege nativ. Acest lucru îl face ideal pentru servere care trebuie să proceseze mii de știri simultan cu un consum minim de curent și resurse.
- **Fără "Garbage Collector":** Spre deosebire de limbaje precum Java sau C#, Rust nu are nevoie de un proces secundar care să "curețe" memoria în timpul rulării, ceea ce elimină pauzele neașteptate în execuție.

```
1  fn main() {
2      let mesaj = String::from("Salut");
3      let copie = mesaj; // Proprietatea asupra textului se muta la
   'copie'
4
5      // Urmatoarea linie ar opri compilarea in Rust:
6      // println!("{}", mesaj);
7      // Motivul: 'mesaj' nu mai detine datele, prevenind
   utilizarea dubla a memoriei.
8  }
```

Listing 1: Exemplu de rigoare in Rust

3.2 Limbajul de programare Python: Limbajul Inteligenței Artificiale

Dacă Rust este despre viteză și precizie, Python este despre **simplitate și flexibilitate**. Este adesea primul limbaj învățat de studenți datorită sintaxei sale care seamănă foarte mult cu limba engleză.

Rolul Python în proiect:

- **Limbaj Interpretat:** Spre deosebire de Rust, Python este un limbaj **interpretat**, ceea ce înseamnă că instrucțiunile sunt citite și executate rând cu rând de către un program special (interpretat). Deși este mai lent la calcule matematice brute, simplitatea sa îl face ideal pentru a coordona biblioteci complexe de calcul.
- **Standardul în Inteligență Artificială:** Aproape toate descoperirile recente în domeniul AI (precum ChatGPT sau recunoașterea facială) sunt dezvoltate în Python. Acest lucru se datorează numărului imens de "pachete" (biblioteci de cod gata scrise) disponibile gratuit.
- **Sintaxă "Curată":** Python elimină nevoia de acolade și punct și virgulă, folosind **indentarea** (spațiile de la începutul rândului) pentru a structura codul, ceea ce forțează programatorul să scrie cod ordonat și ușor de citit.

```
1  # In Python, operatiile complexe devin usor de citit
2  text = "Redy este o platforma inteligenta"
3  cuvinte = text.split() # Imparte textul in cuvinte
4
5  for cuvânt in cuvinte:
6      print(f"Am gasit cuvantul: {cuvânt}")
7
```

Listing 2: Simplitatea Python in procesarea textului

3.3 Biblioteca React: Construirea Interfețelor Moderne

React nu este un limbaj de programare propriu-zis, ci o bibliotecă construită peste **JavaScript** (limbajul care rulează în browser). React a revoluționat modul în care construim site-uri web, trecând de la pagini statice la aplicații web dinamice.

Concepte explicate:

- **Gândirea în Componente:** În loc să scriem o pagină web ca pe un document lung, o vedem ca pe un set de piese de "LEGO". Avem o componentă pentru butoane, una pentru cardul de știri și una pentru meniul de sus. Acestea pot fi refolosite oriunde în aplicație.
- **Actualizare Automată:** În dezvoltarea web clasică, dacă voiai să schimbi un preț pe ecran, trebuia să "cauți" manual elementul și să-l modifiți. În React, programatorul doar descrie cum arată interfața în funcție de date, iar React se ocupă automat de actualizarea ecranului atunci când datele se schimbă (de exemplu, când apare o știre nouă).
- **Interactivitate Fluidă:** Utilizarea React permite paginii să se actualizeze instant, fără a fi nevoie de o reîncărcare completă (refresh), oferind o experiență similară cu o aplicație de telefon mobil.

```
1  function ButonAprecieri() {  
2      const [aprecieri, setAprecieri] = React.useState(0);  
3  
4      return (  
5          <button onClick={() => setAprecieri(aprecieri + 1)}>  
6              Voturi: {aprecieri}  
7          </button>  
8      );  
9  }  
10
```

Listing 3: Exemplu de componenta React

3.4 Comunicarea asincronă între servicii folosind NATS JetStream

Pentru a evita dependențele directe între componente și pentru a crește scalabilitatea aplicației, sistemul utilizează **NATS JetStream** ca broker de mesaje și mecanism de event streaming. Comunicarea asincronă permite serviciilor să funcționeze independent, fără blocarea execuției sau apeluri HTTP directe între ele.

Avantajele utilizării JetStream sunt decuplarea serviciilor (*loose coupling*); toleranță la erori și retry automat; persistența mesajelor; procesare paralelă a sarcinilor; posibilitatea scalării independente a workerelor.

Fluxul de procesare:

1. Backend-ul primește o cerere din partea utilizatorului.
2. Se generează un eveniment ce conține datele necesare procesării.
3. Evenimentul este publicat într-un subject NATS.
4. Un worker ML abonat la acel subject consumă mesajul.
5. După procesare, worker-ul poate publica un nou eveniment cu rezultatul.

Mecanismul Publish/Subscribe:

- **Publish:** un serviciu emite un mesaj fără să cunoască cine îl va procesa.
- **Subscribe:** unul sau mai multe servicii ascultă acel canal și reacționează la evenimente.

```
1  use async_nats::jetstream;
2  use serde::{Serialize, Deserialize};
3  use uuid::Uuid;
4
5  #[derive(Serialize, Deserialize)]
6  pub struct WorkerTaskPayload {
7      pub article_uuid: Option<Uuid>,
8      #[serde(flatten)]
9      pub extra: serde_json::Map<String, serde_json::Value>,
10 }
11
12 async fn publish_task(
13     js: &jetstream::Context,
14     article_id: Uuid
15 ) -> Result<(), anyhow::Error> {
16     let mut headers = async_nats::HeaderMap::new();
17     // ID unic pentru deduplicare: subiect:uuid
18     headers.insert("Nats-Msg-Id", format!("ml.sentiment:{}",
19 article_id).as_str());
20
21     let payload = WorkerTaskPayload {
22         article_uuid: Some(article_id),
23         extra: serde_json::Map::new(),
24     };
```



```

25     let payload_bytes = serde_json::to_vec(&payload)?;
26
27     // Publicarea evenimentului în stream-ul JetStream
28     js.publish_with_headers(
29         "tasks.ml.sentimental-analysis".to_string(),
30         headers,
31         payload_bytes.into()
32     ).await?;
33
34     Ok(())
35 }
36

```

Listing 4: Publicarea unei sarcini ML în JetStream (Rust)

```

1  import nats
2  import json
3
4  async def message_handler(msg):
5      # Decodificarea sarcinii primite
6      payload = json.loads(msg.data.decode())
7      article_uuid = payload.get("article_uuid")
8
9      print(f"Procesare articol: {article_uuid}")
10
11     try:
12         # Logica de analiză AI (sentiment, categorie)
13         await perform_ml_analysis(article_uuid)
14         # Confirmarea procesării (Acknowledge)
15         await msg.ack()
16     except Exception:
17         # Reîncercare automată în caz de eșec
18         await msg.nak(delay=60)
19
20     async def run_worker():
21         nc = await nats.connect("nats://localhost:4222")
22         js = nc.jetstream()
23
24         # Abonare la subiectele ML cu un consumator durabil
25         await js.subscribe(
26             "tasks.ml.*",
27             cb=message_handler,
28             durable="ml-worker",
29             manual_ack=True
30         )
31

```

Listing 5: Worker ML care procesează sarcinile (Python)

Prin această arhitectură bazată pe evenimente, backend-ul nu trebuie să aștepte finalizarea procesării AI. Astfel, aplicația rămâne rapidă și responsivă chiar și atunci când anumite operații necesită timp mare de execuție.

4 Detalii tehnice de implementare

Această secțiune descrie în detaliu arhitectura sistemului și soluțiile tehnice adoptate pentru a asigura performanța, securitatea și scalabilitatea platformei Redy.

4.1 Backend-ul în Rust: Performanță și Securitate

Serverul central este pilonul de rezistență al aplicației, fiind construit pe o stivă tehnologică modernă care prioritizează execuția asincronă.

1. Arhitectura API-ului (Axum): Utilizăm framework-ul **axum**, care se bazează pe biblioteca **tokio** pentru runtime-ul asincron. Un aspect tehnic avansat este utilizarea **Extractorilor**, care permit validarea și preluarea automată a datelor din cererile HTTP (starea bazei de date, autentificarea utilizatorului etc.).

2. Securitatea și Motorul de Scraping: O componentă critică este serviciul de extragere a conținutului articolelor. Acesta include măsuri de protecție împotriva atacurilor de tip **SSRF (Server-Side Request Forgery)** prin validarea riguroasă a adreselor IP, interzicând accesul la rețele private sau locale (*loopback*).

```
1 pub async fn get_url_contents(url: &str) -> Result<String> {
2     validate_url(url)?; // Prevenire SSRF
3
4     // Strategie de tip Fallback pentru a pacali protectiile anti
5     -bot
6     if let Ok(html) = normal::fetch(url).await { return Ok(html); }
7     if let Ok(html) = googlebot::fetch(url).await { return Ok(
8         html); }
9     if let Ok(html) = amp::fetch(url).await { return Ok(html); }
10
11     Err(anyhow!("Failed to fetch content from all sources"))
12 }
```

Listing 6: Validarea URL-urilor și motorul de scraping cu fallback

3. Gestiunea Datelor (SeaORM): Interacțiunea cu PostgreSQL este realizată printr-un ORM asincron (**SeaORM**). Acesta ne permite să utilizăm **ActiveModels** pentru operații de tip CRUD sigure și migrații de bază de date versionate, scrise tot în Rust, asigurând consistența schemei între medii diferite.

4.2 Procesarea ML în Python: Inteligență Asincronă

Componenta de Machine Learning este proiectată să funcționeze ca un set de workeri independenți, conectați la backend prin NATS JetStream.

1. Managementul Modelelor: Pentru a optimiza consumul de memorie (modelele pot ocupa peste 2GB RAM), am implementat un sistem de încărcare de tip **Lazy Loading/Singleton**. Modelele sunt încărcate o singură dată la pornirea worker-ului și sunt partajate între toate sarcinile de procesare primite.

2. Procesul de Clasificare (Zero-Shot): Spre deosebire de clasificarea tradițională care necesită re-antrenare pentru categorii noi, Redy utilizează tehnica *Zero-Shot Classification*. Acest lucru permite sistemului să categorizeze știri în orice domeniu nou definit de utilizator, doar prin furnizarea numelui categoriei ca etichetă (label).

```
1  def handle_categorize(article_uuid: uuid.UUID):
2      # Preluam toate categoriile disponibile din sistem
3      categories = repository.get_all_human_category_names()
4      text = utils.extract_text_from_html(article.html_content)
5
6      # Modelul calculeaza probabilitatea pentru fiecare eticheta
7      result = model.classify(text, categories)
8      top_category = result['labels'][0] # Cea mai probabila
9      categorie
10
11     repository.update_article_category(article_uuid, top_category)
```

Listing 7: Logica de clasificare dinamica

3. Reziliența prin Mesagerie: Utilizarea `nats-py` cu confirmări manuale (`manual_ack=True`) asigură că nicio știre nu rămâne neprocesată. Dacă un model ML eșuează din cauza lipsei de resurse, mesajul este retrimis automat către un alt worker disponibil.

4.3 Frontend-ul în React: Interfață Reactivă și Tipizată

Clientul web este o aplicație modernă construită cu TypeScript, oferind o experiență similară unei aplicații native.

1. Managementul Stării (TanStack Query): În loc să gestionăm manual `useEffect` și `loading states`, utilizăm **TanStack Query**. Această bibliotecă oferă un mecanism robust de *caching*, *background refetching* și *synchronization* între client și server.

```
1      export function FeedArticleList({ feedUuid }) {
2          // Cererea este cache-uita si re-executata doar daca datele
3          devin "stale"
4          const { data, isLoading } = $api.useQuery("get", "/articles",
5          {
6              params: { query: { feed_uuid: feedUuid } }
7          });
8
9          if (isLoading) return <LoadingSkeleton />;
10
11         return (
12             <div className="grid gap-4">
13                 {data.map(item => <ArticleCard key={item.id} {...item} />)}
14             </div>
15         );
16     }
```

Listing 8: Sincronizarea automata a datelor cu TanStack Query

2. Integrarea cu API-ul și TypeScript: Utilizăm un generator de tipuri care transformă definiția OpenAPI a backend-ului în interfețe TypeScript. Acest lucru garantează că orice schimbare în structura datelor de pe server va fi detectată la compilarea frontend-ului, eliminând erorile de tip "undefined field".

3. Componente UI (Shadcn/UI): Interfața este construită folosind un sistem de componente bazat pe **Radix UI** și **Tailwind CSS**, asigurând accesibilitate maximă și o estetică modernă (Dark/Light mode) fără a compromite performanța de randare.

5 Resurse hard și soft necesare

5.1 Resurse Soft

- **Limbaje:** Rust 1.75+, Python 3.10+, Node.js 18+.
- **Baze de date:** PostgreSQL 14+.
- **Infrastructură:** NATS Server, Docker.

5.2 Resurse Hard

- **Procesor:** Minim 2 nuclee (recomandat 4 pentru ML).
- **RAM:** Minim 4GB (modelele ML pot fi intensive).
- **Stocare:** 10GB pentru baza de date și containere.

6 Modalități de utilizare

6.1 Instalare și Pornire

Aplicația folosește Docker Compose pentru o pornire rapidă:

1. Clonarea proiectului.
2. Comanda: `docker-compose up -d`.
3. Accesare: `http://localhost:3000`.

6.2 Fluxul Utilizatorului

Utilizatorul se autentifică, adaugă un flux RSS de interes (ex: știri tehnice), iar sistemul începe automat fetch-ul și analiza. Articolele apar în dashboard cu etichete de sentiment și categorie.

7 Posibilități de dezvoltare

- **Sistem de Recomandări:** Bazat pe istoricul de citire.
- **Aplicație Mobilă:** Dezvoltată în React Native.
- **Suport Multilingv:** Extinderea modelelor ML pentru limba română.